



Audio tömörítő egység megvalósítása FPGA-val

Önálló laboratórium 2. zárójegyzőkönyv
2025/26. I. félév

Nyiri Levente (ULYHQH)

Villamosmérnök szakos hallgató
MSc FPGA alapú rendszerek mellékspecializáció

Konzulens:

Szántó Péter

(Mesterséges Intelligencia és Rendszertervezés Tanszék)

Feladatkiírás

Az AMD Ultrascale+ MPSoC FPGA-k használata számos multimédia feldolgozást igénylő alkalmazásban kedvező opció.

Míg a video tömörítésre az eszköz dedikált hardveregységet tartalmaz, addig az audio kezelése elsősorban az általános célú ARM processzoros alrendszeren oldható meg. Ez azonban fogyasztás, illetve CPU terhelés szempontjából nem feltétlenül optimális.

A feladat célja, az alkalmazások jelentős részében előkerülő audiotömörítésre egy, az FPGA általános erőforrásait felhasználó hardver gyorsító kifejlesztése, valamint ennek összehasonlítása a tisztán szoftveres megoldással.

Tartalomjegyzék

1. Bevezetés	1
2. Technológiai háttér	1
3. Encoder	1
3.1. Az encoder működési elve	1
3.2. Frame struktúra	2
3.2.1. Header	2
3.2.2. Side information	3
3.2.3. Main Data	6
3.3. Analysis Polyphase Filterbank	7
3.4. FFT tervezés	8
4. Összefoglalás	16
4.1. Értékelés	16
4.2. Folytatás	16

1. Bevezetés

A dolgozat célja bemutatni egy FPGA alapú audio tömörítő rendszer tervezési lépéseit. A munkát ebben a félévben kezdtem, és a feladat komplexitásából kifolyólag várhatóan a Diplomatervezés 2. tárgy keretén belül fogom befejezni.

2. Technológiai háttér

Az első fontos döntés, amit meg kell hozni, az a tömörítéshez használt szabvány kiválasztása. Az MPEG-1 Audio és az MPEG-2 AAC szabványok összevetése után az MPEG-1 Audio Layer III (MP3) implementálása mellett döntöttem.

Bár az AAC egy modernebb szabvány, az MP3 továbbra is széles körben használatban van, viszont több segédanyag található hozzá, ami megkönnyíti a fejlesztést.

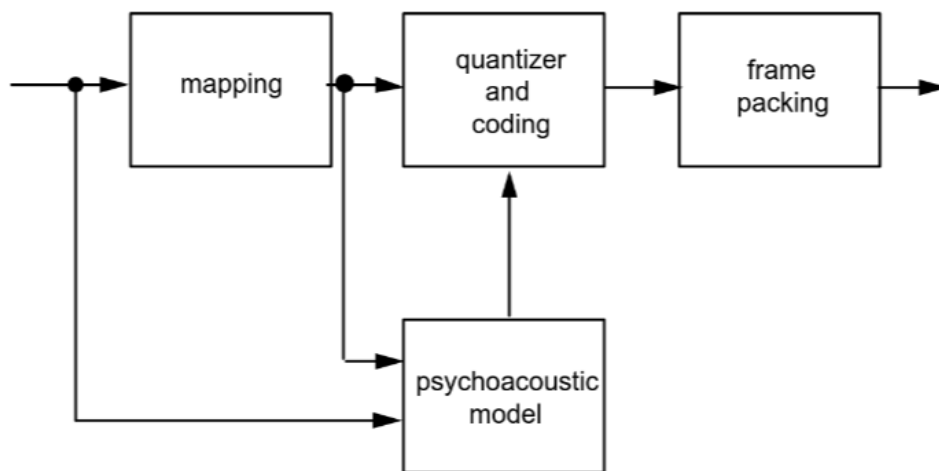
Több open source MP3 encoder is létezik, a legnépszerűbb a LAME, ezt fogom használni, hogy összevessem az FPGA-s megvalósítást a szoftveressel.

A szabvány megértéséhez többek közt a LAME forrás kódját is olvastam, illetve lefordítottam és kipróbáltam egy teszt .wav file-al.

3. Encoder

3.1. Az encoder működési elve

Az encoder egyszerűsített blokkvázlata[1] az 1. ábrán látható.



1. ábra. Encoder blokkvázlat

Az MP3 tömörítés folyamata a következő lépésekből áll. Először egy filter bank segítségével frekvenciasávokra osztjuk a jelünket (mapping), ezzel párhuzamosan egy pszichoaakusztikus modell alapján meghatározzuk, hogy mi a legnagyobb megengedhető zajszint

az egyes sávokban. Ezek segítségével meghatározzuk, hogy az egyes sávokat hány bittel kódoljuk, és végül egy bitstreambe formázzuk a kódolt biteket.

3.2. Frame struktúra

A kimeneti bitstream kisebb, önállóan dekódolható egységekre van tagolva, ezeket frame-eknek hívjuk. Minden frame 1152 audio mintát tartalmaz. Egy frame továbbá szét van választva 2 granule-ra, mindkettőre 576 minta jut. Egy frame méretét byteban a(z) 1. egyenlet írja le.

$$\text{frame size} = \frac{144 \cdot \text{bitrate}}{f_s} + \text{Padding} \quad (1)$$

Egy frame felépítése a(z) 1. táblázatban [2] látható.

Header	CRC	Side_Information	Main_Data	Ancillary_Data
--------	-----	------------------	-----------	----------------

1. táblázat. Az MPEG-1 Audio frame mezői

3.2.1. Header

A header tartalmazza a szinkronizációs biteket és egyéb információkat a frame-ről, felépítését mutatja a(z) 2. ábra [2].

	Sync		
ID	Layer	Prot. bit	
	Bitrate		
Frequency	Pad. bit	Priv. bit	
Mode	Mode extension		
Copy Home	Emphasis		

2. ábra. Frame header

Sync (12 bit) Szerepe a szinkronizálás, mind a 12 bitnek 1-esnek kell lennie: `sync = 12'b1111_1111_1111`

ID (1 bit) Az MPEG verziót határozza meg (MPEG-1 vagy MPEG-2)

Layer (2 bit) Layer I, II vagy III

Protection bit (1 bit) Meghatározza, használunk-e CRC-t

Bitrate (4 bit) A bitrate beállítása

Frequency (2 bit) A mintavételi frekvencia meghatározása

Padding bit (1 bit) Néhány frame-nek szüksége van rá, hogy a bitrate pontos legyen

Private bit (1 bit) Applikáció-specifikus trigger

Mode (2 bit) Csatornamód

Mode extension (2 bit) Csak joint stereo esetén használatos, további specifikáció

Copyright bit (1 bit) Jelzi, ha a tartalom szerzői jogi védelem alatt áll

Home bit (1 bit) A frame az eredeti adathordozón található-e

Emphasis (2 bit) A dekódernek szükséges-e de-emphasist alkalmaznia zajcsökkentés után

Megkötések:

Mivel egy általános, minden opciót támogató encoder túlmutat a feladat keretein, a tervezés során a(z) 2. táblázatban látható fix konfigurációs beállításokat alkalmazom a Header mezőire vonatkozóan.

Mező Neve	Bit Érték	Jelentés
ID	1	MPEG-1
Layer	01	Layer III
Protection bit	1	Használunk CRC-t
Bitrate	1110	320 Kbps
Frequency	00	44.1 kHz mintavételi frekvencia
Mode	00	Stereo
Emphasis	00	Nem kell de-emphasis dekódolásnál, nem használunk noise suppression-t

2. táblázat. Megkötések a bitekre

3.2.2. Side information

A side information további információt tartalmaz arra vonatkozóan, hogy hogyan kell dekódolni a frame-et. A felépítését a(z) 3. táblázat mutatja.

main_data_begin	private_bits	scfsi	Side_info gr. 0	Side_info gr. 1
-----------------	--------------	-------	-----------------	-----------------

3. táblázat. A side information mezői

main_data_begin (9 bit) Layer III-nál bit reservoir használatával egy adott frame main data helyén szabadon maradt helyet másik frame-ek is használhatják. Ez a mező azt a negatív offsetet adja meg, amennyivel korábban kezdődik egy frame-nek a main data-ja a sync word-höz képest.

private_bits (5 bit) Szabad felhasználás

scfsi (4-4 bit) ScaleFactor Selection Information, azt határozza meg, hogy az egyes scalefactor-ok mindkét granule-ra használhatóak-e vagy külön kell mindkettőre küldeni. Ha elég csak egyet küldeni, azzal biteket nyerünk, amit Huffman kódoláshoz lehet használni.

Layer III-nál a scalefactorok 4 csoportba vannak osztva a(z) 4. táblázat szerint. Mindkét csatornára vonatkozóan, ha egy bit 1-esben van, akkor az adott csoporthoz tartozó scalefactorokat a frameben lévő második granule újra fogja használni. Ha short window-t használunk (`block_type == 2`) bármely granule-ban, akkor a scalefactorokat mindig külön küldjük mindkét csatornára.

group	scalefactor bands
0	0,1,2,3,4,5
1	6,7,8,9,10
2	11,12,13,14,15
3	16,17,18,19,20

4. táblázat. Scalefactor csoportok

A granule-okhoz tartozó `side_info` még további mezőkből áll, ezt mutatja a(z) 5. táblázat.

<code>part2_3_length</code>	<code>big_values</code>	<code>global_gain</code>	<code>scalefac_compress</code>
<code>windows_switching_flag</code>	<code>block_type</code>	<code>mixed_block_flag</code>	<code>table_select</code>
<code>subblock_gain</code>	<code>region0_count</code>	<code>region1_count</code>	<code>preflag</code>
<code>scalefac_scale</code>	<code>count1table_select</code>		

5. táblázat. Granule side information mezői

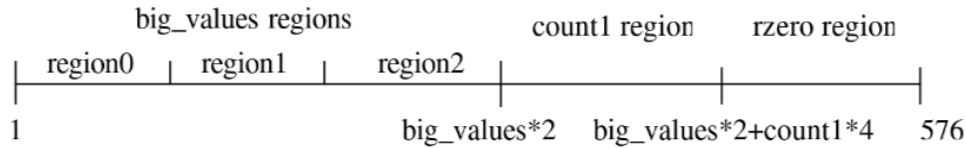
par2_3_length (12-12 bit) Megmondja, hogy a main data részében a frame-nek hány bit van allokalva scalefactor-oknak (part2) és Huffman kódolt adatnak (part3).

big_values (9-9 bit) Az egyes granule-ok spektrális komponensei más Huffman kód táblázatokkal vannak kódolva. A teljes spektrum 0-tól a Nyquist frekvenciáig több részre van bontva, és ezek a részek máshogy vannak kódolva. A partícionálás a maximális kvantált értékek alapján történik.

Megszámoljuk, hogy minden frekvencián összesen hány 0 érték van, ezeknek a számát az `rzero`-ban tároljuk. A `count1` mezőben 4-esével vannak értékek, és az

abszolútértéke nem haladja meg az 1-et. A többi érték a **big_values** mezőben van, ezen belül is 3 részre osztva. Az abszolút maximum értéke ennek a régióknak 8191.

A **big_values** mező felosztása a(z) 3. ábrán látható.



3. ábra. **big_values** felosztása

global_gain (8-8 bit) Kvantálás lépésköz.

scalefac_compress (4-4 bit) Hány bitet használjon a scalefactorok átviteléhez. Hogy az értéke alapján az első és második scalefactor group hány bitet kap az a(z) 6. táblázatban látható.

scalefac_compress	slen1	slen2
0	0	0
15	4	3

6. táblázat. Scalefac compress értékek

windows_switching_flag (1-1 bit) Megmutatja, ha a normálon kívül másmilyen ablaktípus van használatban.

block_type (2-2 bit) Ha nem normál ablakot használunk, mutatja, hogy melyet. A lehetőségek a 7. táblázatban vannak.

block_type	window type
00	forbidden
01	start
10	3 short windows
11	end

7. táblázat. Block type értékek

mixed_blockflag (1-1 bit) Akkor használható, ha a **windows_switching_flag** be van állítva. Azt jelzi, hogy más ablaktípust használunk alacsonyabb és magasabb frekvenciákon.

table_select (10-10 bit) Megadja milyen Huffman kódolást használjunk.

subblock_gain (9-9 bit) Ha **windows_switching_flag** set és **block_type** == 10, akkor a gain offsetet mutatja a **global gain**-tól.

region_address1 (4-4 bit) region_address2 (3-3 bit) A spektrumot további részekre osztjuk a Huffman kódoláshoz, ezek a változók ezeknek a régióknak a kezdőcímét tartalmazzák.

preflag (1-1 bit) További nagyfrekvenciás erősítése a kvantált mintáknak.

scalefac_scale (1-1 bit) A scalefactorok logaritmikusan kvantáltak a(z) 8. táblázat szerint.

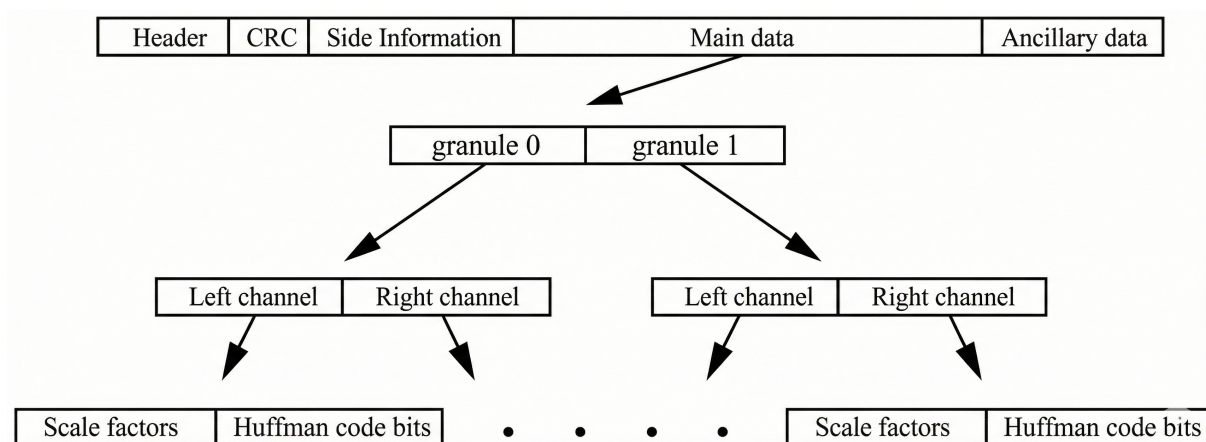
scalefac_scale	step size
0	$\sqrt{2}$
1	2

8. táblázat. Scalefac scale értékek

count1table_select (1-1 bit) Huffman kódolást választ a count1 mezőben lévő értékekhez.

3.2.3. Main Data

Scalefactorokból, Huffman kódolt bitekból és ancillary data-ból áll. Az elrendezés a(z) 4. ábrán látható.



4. ábra. Main data, granule-ok és channelek elrendezése

Scale factors A céljuk, hogy redukálják a kvantálási zajt. Ha a minták egy adott scalefactor band-ben megfelelően vannak scale-elve, akkor a kvantálási zaj teljesen ki lesz maszkolva.

Huffman code bits Itt találhatóak a Huffman kódolt bitek.

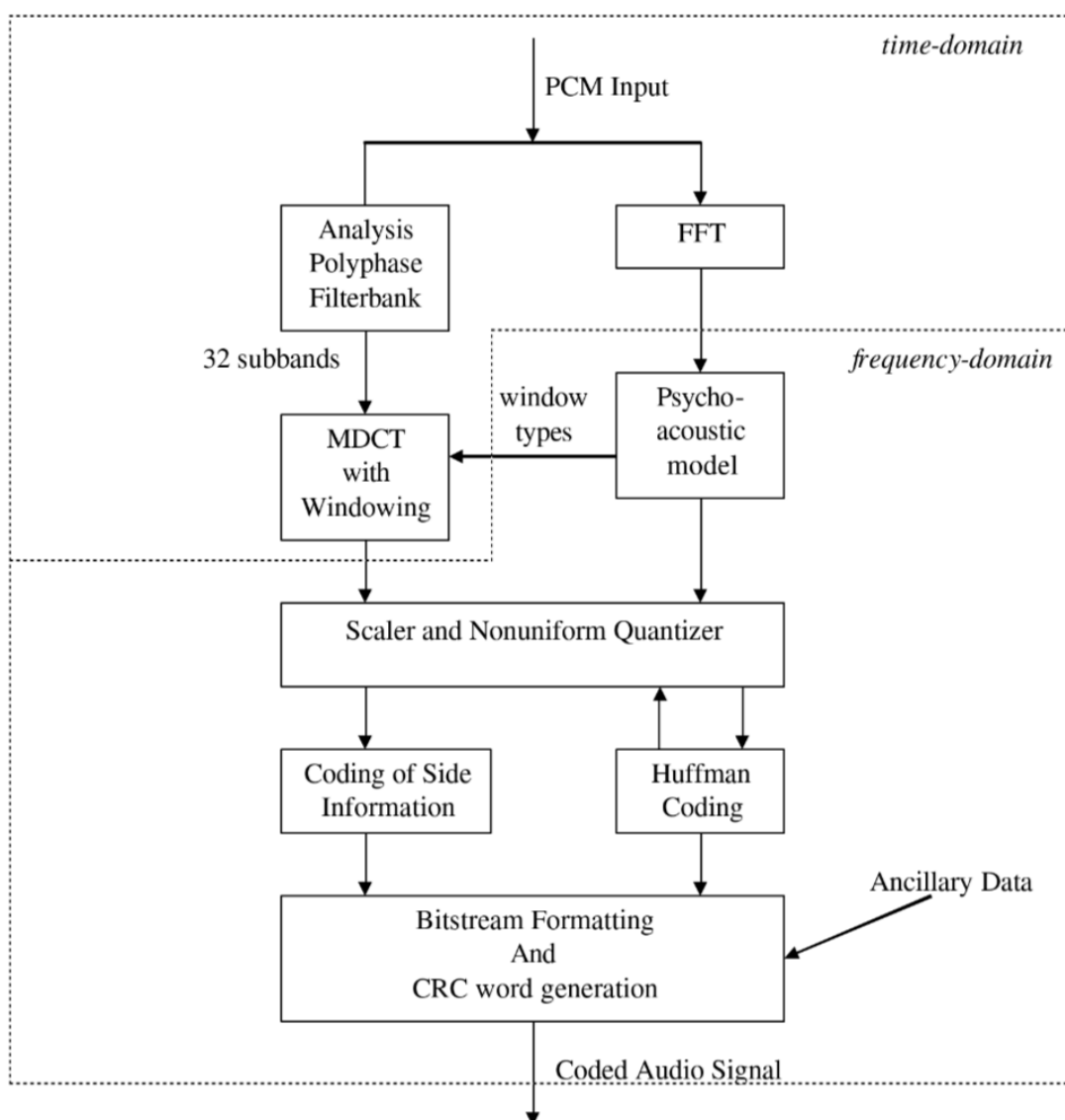
Ancillary data Opcionális, felhasználó definiálhatja.

3.3. Analysis Polyphase Filterbank

A(z) 5. ábrán látható egy részletesebb blokkvázlat az encoderról. Az Analysis Polyphase Filterbankba megy a PCM input, itt az 1152 PCM minták mindegyike 32 egyenletesen eloszló frekvenciasávba lesz szétosztva. Mivel $f_s = 44.1$ kHz ezért a Nyquist frekvencia $f_{Nyquist} = 22.05$ kHz. Minden sáv

$$\frac{22050}{32} = 689.0625 \text{ Hz}$$

széles lesz. Mivel minden mintát 32 subband-re osztunk, ezért az eredeti 1152-ből $1152 * 32 = 36864$ mintánk lesz. A folyamat végén viszont 32-vel decimálunk minden subband-et, így ismét 1152 mintánk lesz.



5. ábra. Encoder blokkvázlat részletesebben

3.4. FFT tervezés

A bejövő adatnak a spektrumát ismerünk kell a pszichoakusztikus modell alkalmazásához, ezért FFT-t hajtunk végre a bejövő jelen, a filterbankkal párhuzamosan.

A művelet elvégzéséhez a Xilinx FFT IP core-ját fogom használni. Meg kell határozni, hogy az elérhető architektúrák közül melyik a legkisebb erőforrás igényű amely megfelel a célunknak. Layer III-nál két FFT-t kell végezni párhuzamosan. Block length == window length.

Short blocks shift length = 192 samples, window length = 256 samples

Long blocks shift length = 576 samples, window length = 1024 samples

Mivel $f_s = 44.1$ kHz, ezért

$$window_size_1 = \frac{256}{44100} = 5.8ms$$

$$window_size_2 = \frac{1024}{44100} = 23.2ms$$

$$frequency_resolution_1 = \frac{44100}{256} = 172.3 \text{ Hz}$$

$$frequency_resolution_2 = \frac{44100}{1024} = 43.1 \text{ Hz}$$

Látszik, hogy a kisebb ablakok jobb felbontást biztosítanak időtartományban, a hosszabb ablakok pedig frekvenciatartományban. A cél megtalálni az AMD által biztosított FFT core-nak azt a felparaméterezését, amely a lehető legkisebb erőforrás felhasználás nélkül teljesíteni tudja a követelményeket.

Long blocks:

- Shift length: 576 samples
- FFT méret: 1024 points
- Overlap: $1024 - 576 = 448$ samples (43.75%)

Frame N : samples $[0 \dots 1023]$

Frame $N + 1$: samples $[576 \dots 1599]$

Frame $N + 2$: samples $[1152 \dots 2175]$

Short blocks:

- Shift length: 192 samples
- FFT méret: 256 points
- Overlap: $256 - 192 = 64$ samples (25%)

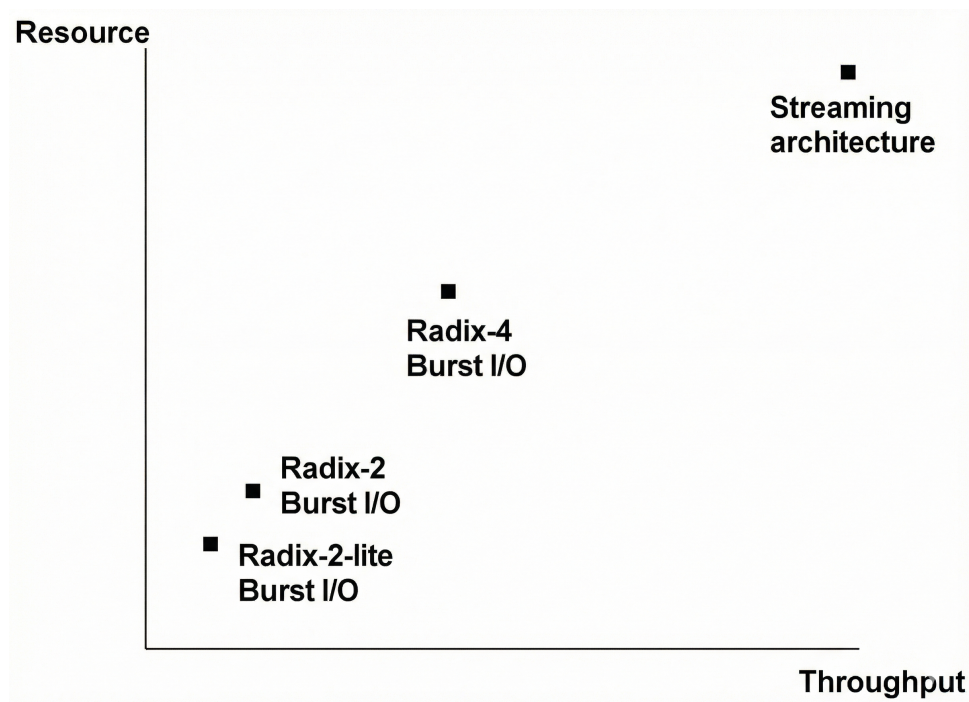
Frame N : samples $[0 \dots 255]$

Frame $N + 1$: samples $[192 \dots 447]$

Frame $N + 2$: samples $[384 \dots 639]$

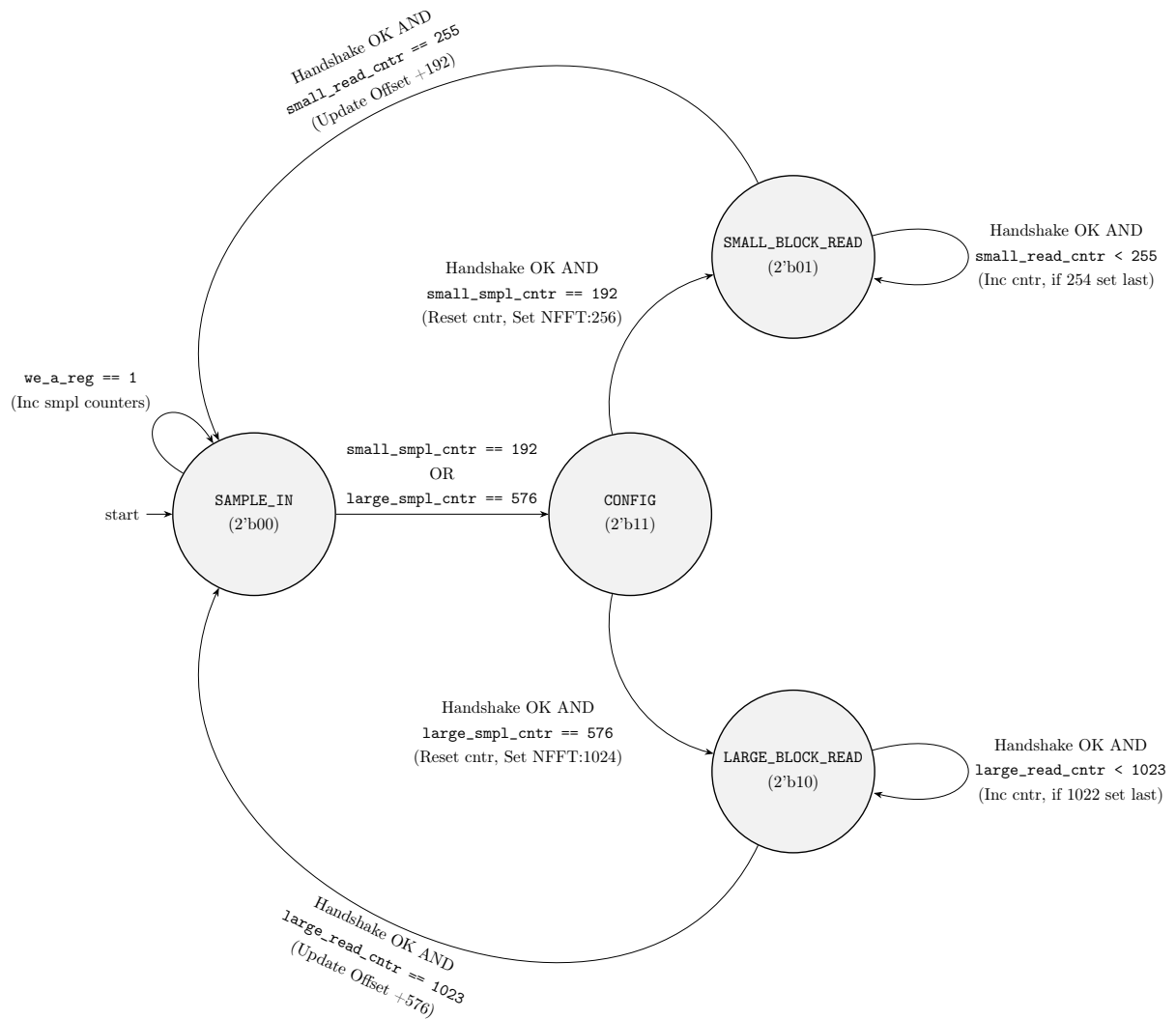
Hosszú ablakoknál $\frac{576}{44100} = 13.06ms$ időbe telik, amíg bejön elegendő adat egy FFT végrehajtására, rövidnél $\frac{192}{44100} = 4.35ms$.

A Xilinx FFT IP core-ban 4 architektúra közül lehet választani, ezeknek az erőforrás-igényét a throughput függvényében mutatja a(z) 6. ábra.



6. ábra. Xilinx LogiCORE IP FFT architektúrák [3]

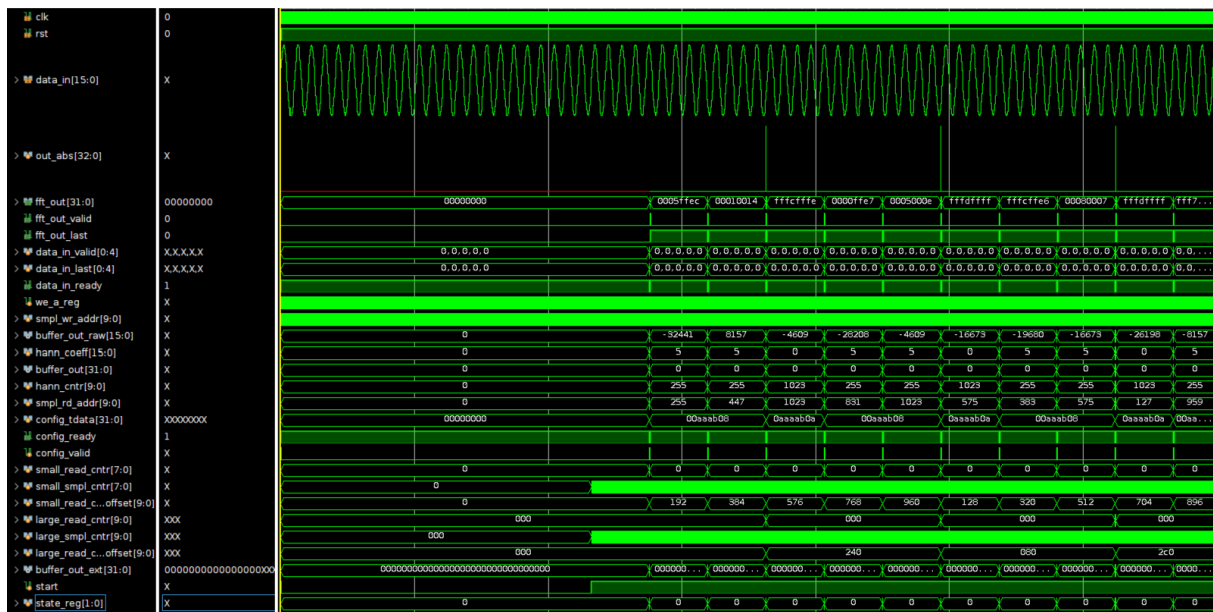
A logikámat a(z) 7. ábrán látható állapotgép vezérli.



7. ábra. Az FFT vezérlő állapotgép diagramja

A `SAMPLE_IN` állapotban írjuk a bejövő mintákat a BRAM-ba, számlálókkal vannak számon tartva, hogy mennyi jött be (külön számlálók a 256 és 1024 pontos FFT-hez), és amint van elegendő egy FFT elvégzéséhez, akkor átmegyünk a `CONFIG` állapotba. Itt megvizsgáljuk, hogy melyik számláló hatására léptünk ide, és konfiguráljuk az FFT core-t. Ezután vagy a `SMALL_BLOCK_READ` vagy a `LARGE_BLOCK_READ` állapotba lépünk, és elvégezzük az adatátvitelt az IP core bemenetére, majd visszatérünk a `SAMPLE_IN` állapotba.

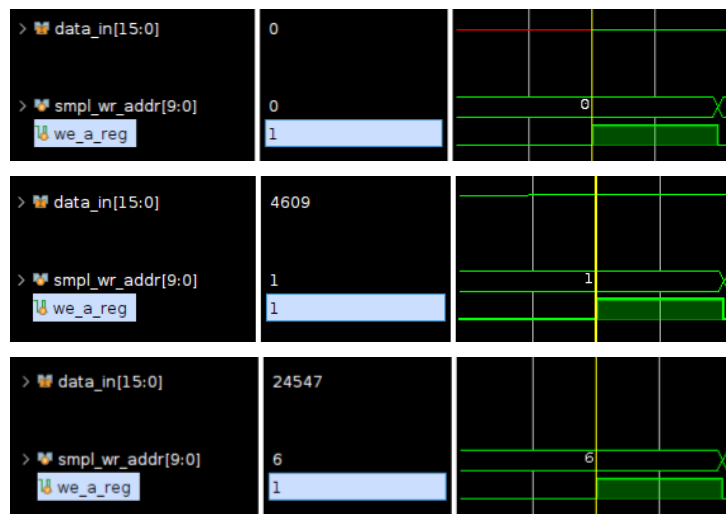
Olvasás után, még mielőtt az FFT bemenetére adjuk a jelet végzünk rajta egy Hann ablakozást, hogy elkerüljük a spektrális szivárgást. Ehhez a Hann együtthatók egy python scripttel lettek generálva és egy `.mem` file-ban vannak tárolva. A feladat folytatása során itt is konkretizálni kell, hogy egy 18k-s BRAM-ba legyenek tárolva.



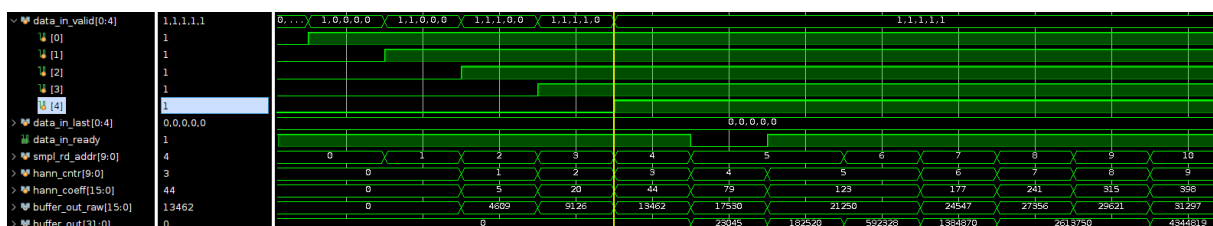
9. ábra. Szimuláció

1kHz-s szinuszjel a bemenet, `we_a_reg` a 44.1kHz-s mintavételi engedélyező jel, ahol pedig `fft_out_valid` tűskét látunk, ott van FFT kimenetünk.

Hogy bizonyosak lehessünk benne, hogy az történik, amit szeretnénk, érdemes megvizsgálni a szorzások eredményét, és az engedélyező jelet.



10. ábra. BRAM-ba beírt adatok



11. ábra. BRAM-ból olvasás, Hann ablakozás

A bufferelt bemeneti értékeket és a hozzájuk tartozó számított kimeneti adatokat a 9. táblázat foglalja össze. Az értékeket összevetve a 10. ábrán látható adatokkal látszik, hogy megfelelően működik.

Index	buffer (bemenet)	Hann-együttható	buffer_out (elvárt)
0	0	0	$0 \times 0 = 0$
1	4609	5	$4609 \times 5 = 23045$
6	24547	177	$24547 \times 177 = 4344819$

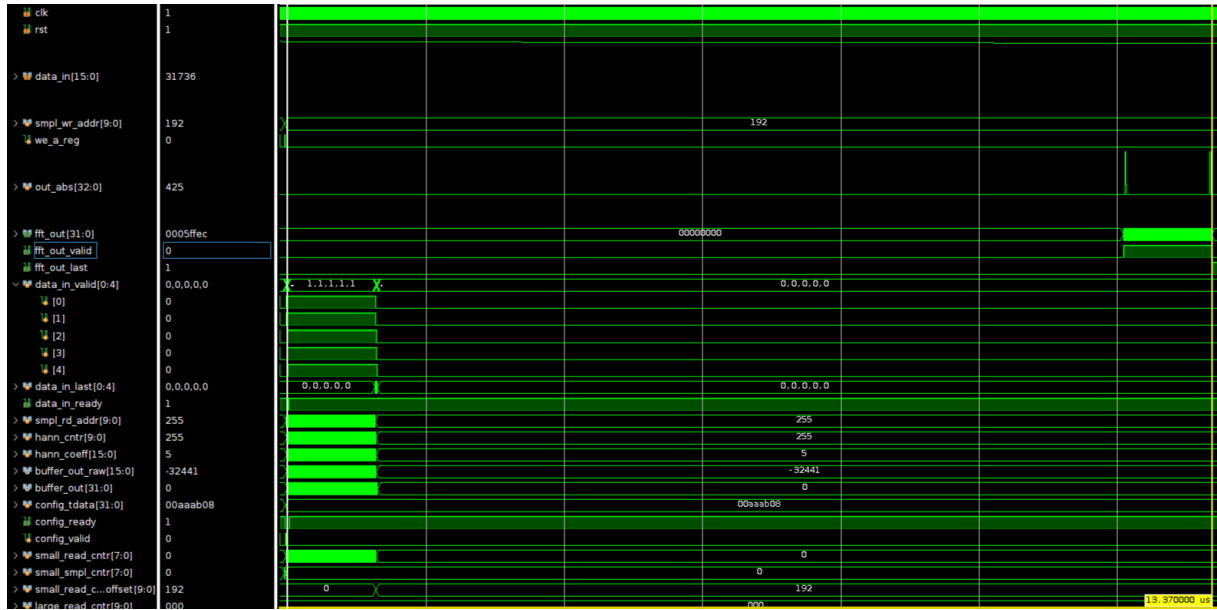
9. táblázat. Bemeneti minták és a Hann-ablakozás utáni elvárt kimeneti értékek

A feldolgozási lánc időzítése kritikus az FFT szempontjából. A pipeline késleltetés az alábbi lépésekből tevődik össze (lásd: 11. és 8. ábra):

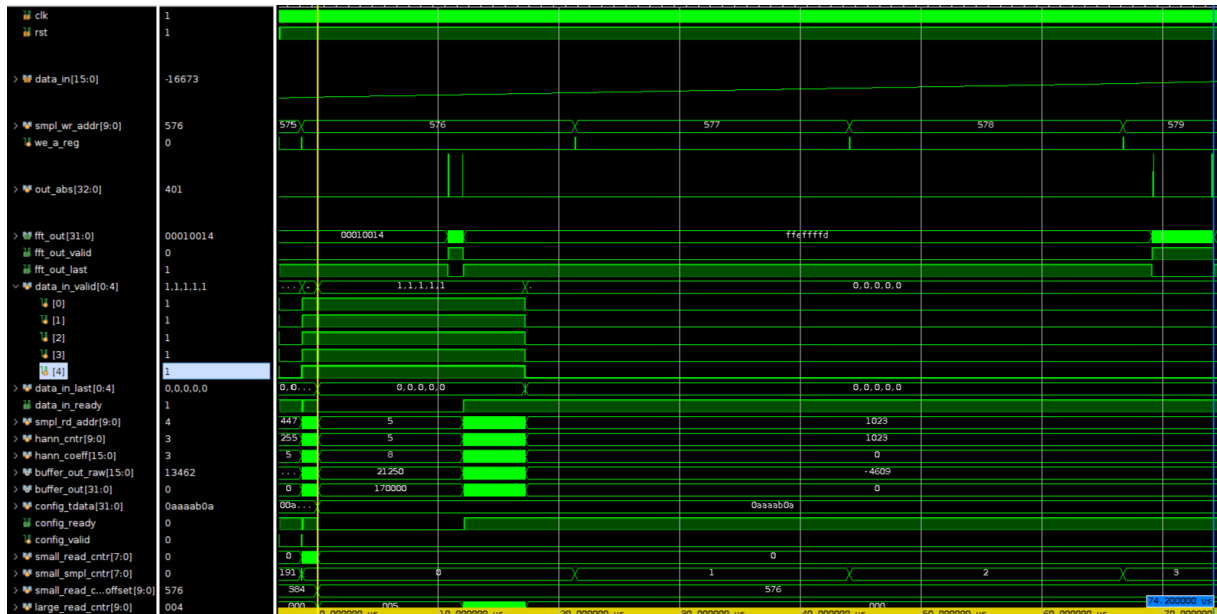
- **T0:** A `data_in_valid[0]` jel magasba váltásakor megkezdődik a kiolvasás a BRAM-ból.
- **T1:** Egy órajelciklussal később után az adat megjelenik a `buffer_out_raw` buszon, és elkezdjük a szorzást a Hann-együtthatóval.
- **T2–T4:** A DSP slice belső pipeline regiszterei további 3 órajelnyi késleltetést okoznak.
- **T4:** A szorzat érvényessé válik a kimeneten, a `data_in_valid[4]` az FFT-nek a `T_VALID` jele.

Az IP core az első vétel után a `T_READY` jelet lehúzza, és amíg készen nem áll a feldolgozásra, addig az első adatot egy buffer-ban tárolja, ez látszik a(z) 11. ábrán is.

Vizsgáljuk meg, hogy mennyi időbe telik egy 256 és egy 1024 pontos FFT végrehajtása. A `data_in_valid[4]` felfutó élétől az `fft_out_valid` lefutó éléig mérjük az időt.



12. ábra. $N=256$ FFT végrehajtási idő



13. ábra. $N=1024$ FFT végrehajtási idő

A(z) 13. ábrán látszik, hogy a 256-os FFT után amíg átkonfigurálódik az IP core 1024-pontosra, addig lehúzza a `data_in_ready`-t, és csak utána folytatódik a feldolgozás.

- Rövid ablakok ($N = 256$):

- Mért végrehajtási idő: $13,37 \mu s$ (12. ábra).
- Rendelkezésre álló idő (192 minta beérkezése): $4,35 ms$.

- Kihasználtság:

$$\eta_{256} = \frac{13,37 \cdot 10^{-6} s}{4,35 \cdot 10^{-3} s} \cdot 100 \approx 0,31\% \quad (2)$$

- **Hosszú ablakok** ($N = 1024$):

- Mért végrehajtási idő: $74,2 \mu s$ (13. ábra).
- Rendelkezésre álló idő (576 minta beérkezése): $13,6 ms$.
- **Kihasználtság:**

$$\eta_{1024} = \frac{74,2 \cdot 10^{-6} s}{13,6 \cdot 10^{-3} s} \cdot 100 \approx 0,55\% \quad (3)$$

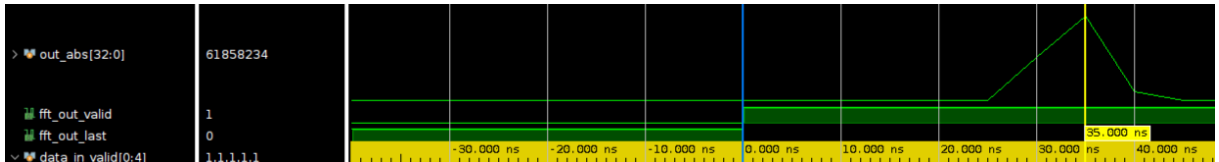
Látszik, hogy hatalmas tartalékunk van, tehát elég lesz egy darab Radix-2-lite Burst I/O architektúrájú FFT IP core időmultiplexáltan használva.

A teszteléshez használt szinusz jel 990.527Hz -s. Szinusz jel spektruma egy darab komponenst tartalmaz egy frekvencián, szimmetrikusan a képzetes tengelyre.

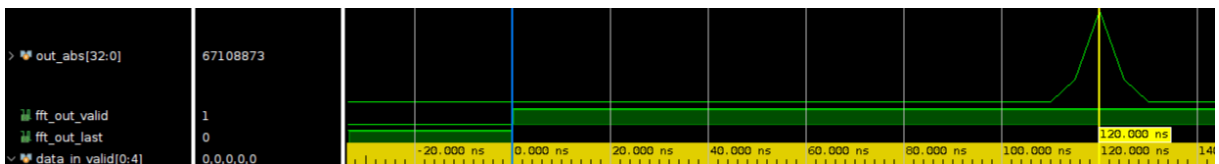
Mivel 200MHz -es az órajelünk, ezért az órajelperiódus $T=5 \text{ ns}$. Az ábrán mért időkből egy periódust le kell vonni, mert jelenleg az abszolútérték-képzés egy órajelbe telik.

FFT pontszám	Felbontás [Hz]	Eltelt órajel periódusok	Számított frekvencia- komponens [Hz]
256	$\frac{44100}{256} \approx 172,27$	$\frac{30}{5} = 6$	1033,62
1024	$\frac{44100}{1024} \approx 43,07$	$\frac{115}{5} = 23$	990.61

10. táblázat. Spektrális komponens frekvenciája



14. ábra. $N=256$ FFT kimeneti komponens frekvenciája



15. ábra. $N=1024$ FFT kimeneti komponens frekvenciája

Ha a felbontásnak nem a kerekített értékével szoroznánk, akkor még jobban visszaadná a számított frekvencia az eredeti jelét, de így is látszik, hogy az történik, amire számítunk.

4. Összefoglalás

4.1. Értékelés

A félév folyamán megismerkedtem az audio tömörítő codec-ek világával, mélyebben az MPEG-1 Audio Layer III (MP3)-al.

Mivel ebben a félévben kezdtem a témát, ezért jelentős mennyiségű idő irodalomkutatással telt. HDL kódban megvalósításra a bejövő jel szabvány szerinti FFT-zése került. Ebből kifolyólag a szoftveres és az FPGA-s megvalósítások összehasonlítására is majd csak a következő félévekben kerül sor.

A téma komplexitásából kifolyólag várható volt, hogy több féléves munka lesz.

4.2. Folytatás

A témát a Diplomatervezés tárgyak keretében fogom folytatni, ahol a következő lépés az FFT kimenetét felhasználva a pszichoakusztikus modell megvalósítása lesz. A végcél egy olyan FPGA-s megvalósítása az MP3 szabványnak, amely valós időben adja ki egy ADC-ről kapott bemenetnek a tömörített bitstream-jét.

Hivatkozások

- [1] ISO/IEC, „Information technology — Coding of moving pictures and associated audio for digital storage media at up to about 1,5 Mbit/s — Part 3: Audio”, International Organization for Standardization, Geneva, Switzerland, International Standard ISO/IEC 11172-3:1993, 1993, (MPEG-1 Audio).
- [2] R. Raissi, „The Theory Behind Mp3”, 2002. cím: <https://api.semanticscholar.org/CorpusID:12806533>.
- [3] Xilinx, *Fast Fourier Transform v9.1 LogiCORE IP Product Guide*, PG109, Vivado Design Suite, Xilinx, 2022. máj. cím: https://www.xilinx.com/support/documents/ip_documentation/xfft/v9_1/pg109-xfft.pdf.
- [4] Xilinx, *7 Series DSP48E1 Slice User Guide*, UG479, v1.10, Xilinx, 2018. márc. cím: https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1.